



RUHR-UNIVERSITÄT BOCHUM

SECURITY

Computer Architecture

System Security

Computer Security

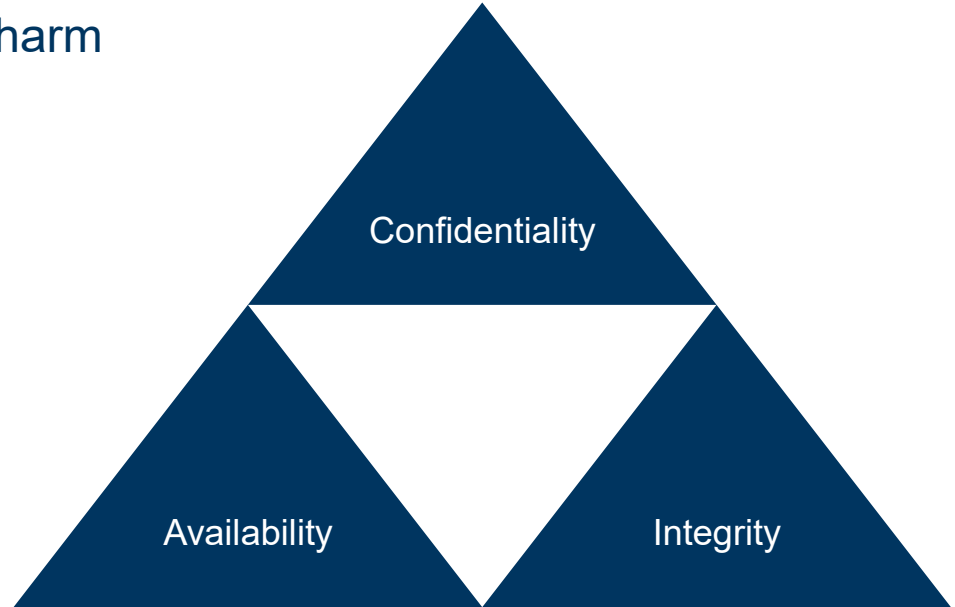
Aim: protect computer systems from harm

Computer system:

- Hardware
- Software
- Data

Harm

- Confidentiality
- Integrity
- Availability



Responsibility for Security

Responsibility for security shared across all system components

- Hardware provides required primitives to enable security
- Software correctly uses the primitives
- Users deploy and use system securely

Hardware support:

- Privilege levels
- Virtual memory

Privilege levels

The CPU provides multiple levels of execution

- User mode
 - User programs
 - Limited instructions
 - Limited (virtual) memory access
- System mode
 - Operating system kernel
 - Access to system management instructions
 - Can change virtual memory maps
- Others – x86 rings, hypervisor / system management

Changing privilege levels

System calls

- Switch from user mode to system mode
- Changes PC and stack to system-defined locations
- Stores original PC and stack to allow restore

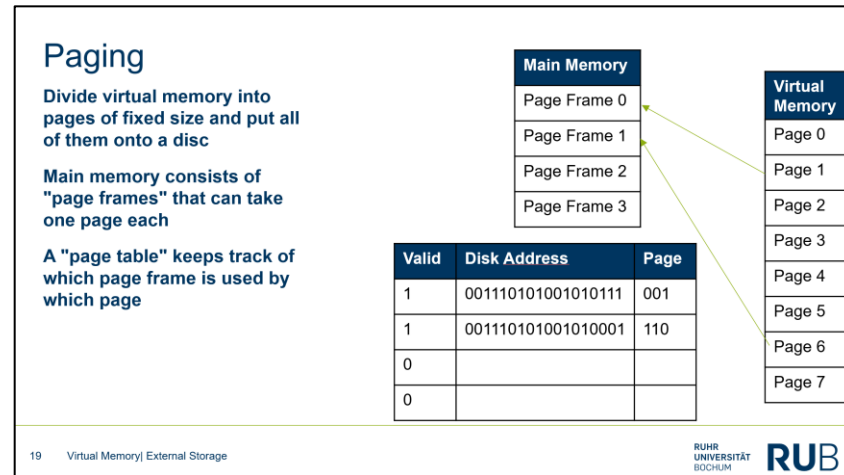
Interrupts

- Change privilege level as part of switch to handler

Virtual memory protection

Virtual memory is a security feature

- Allows mapping of virtual to physical addresses
- Prevents access outside the maps
- Limits access type for mapped pages
 - Read/write
 - Execute
 - Privilege level



Problem: Confidentiality

Caches

CPU vs. Memory



**Processor
Speed**

1 MHz

**Memory
Latency**

500 ns

20×3700 MHz

50 ns

RUHR
UNIVERSITÄT
BOCHUM

RUB

Caches

- Idea:
- The first time the processor needs data it brings it from memory
- It then stores the data nearby for future reuse
- The near storage is called a *cache*

Processor



Cache

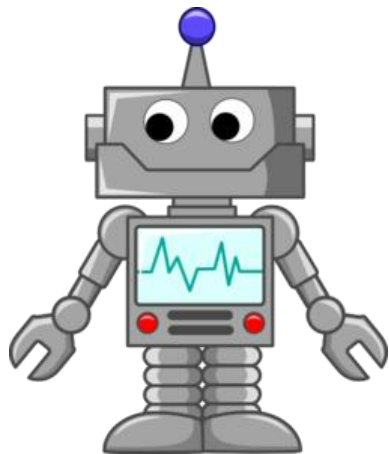


Memory

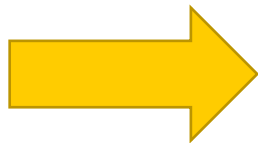
RUHR
UNIVERSITÄT
BOCHUM

RUB

Cache Operation



Program
History

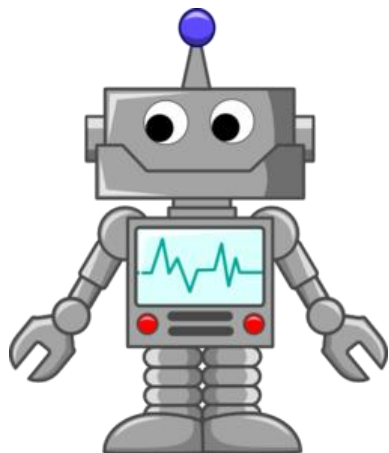


Cache
State

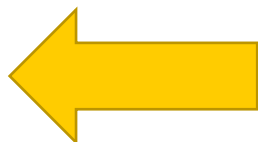


Execution
Time

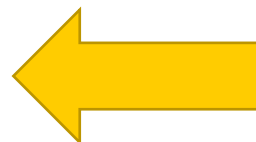
Flipping our view



Program
History



Cache
State



Execution
Time

Cache Consistency

- Memory and cache can be in inconsistent states
 - Rare, but possible
- Solution: Flushing the cache contents
 - Ensures that the next load is served from the memory

Processor



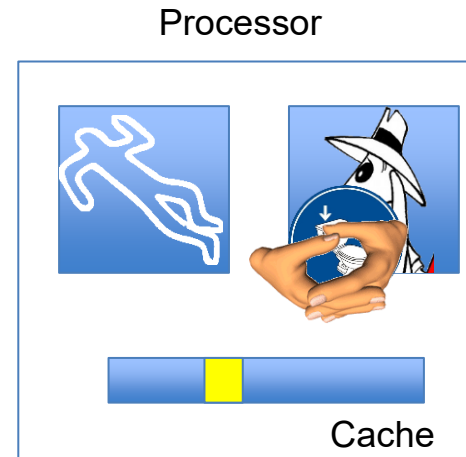
Cache



Memory

The FLUSH+RELOAD Attack

- Flush a memory line from the cache
- Wait a bit
- Measure time to load line
 - slow-> no access
 - fast-> access
- Repeat



Memory

The RSA Encryption System

The RSA encryption is a public key cryptographic scheme



$$M = C^d \bmod N$$

M

$$C = M^e \bmod N$$



Key Generation:

- Select random primes p and q
- Calculate $N = pq$
- Select a public exponent $e (=65537)$
- Compute $d = e^{-1} \bmod \phi(N)$
- (N, e) is the public key
- (p, q, d) is the private key

Square and Multiply Exponentiation

```
 $x \leftarrow 1$   
for  $i \leftarrow |d|-1$  downto 0 do  
   $x \leftarrow x^2 \bmod n$   
  if ( $d_i = 1$ ) then  
     $x = xC \bmod n$   
  endif  
done  
return  $x$ 
```

Example:

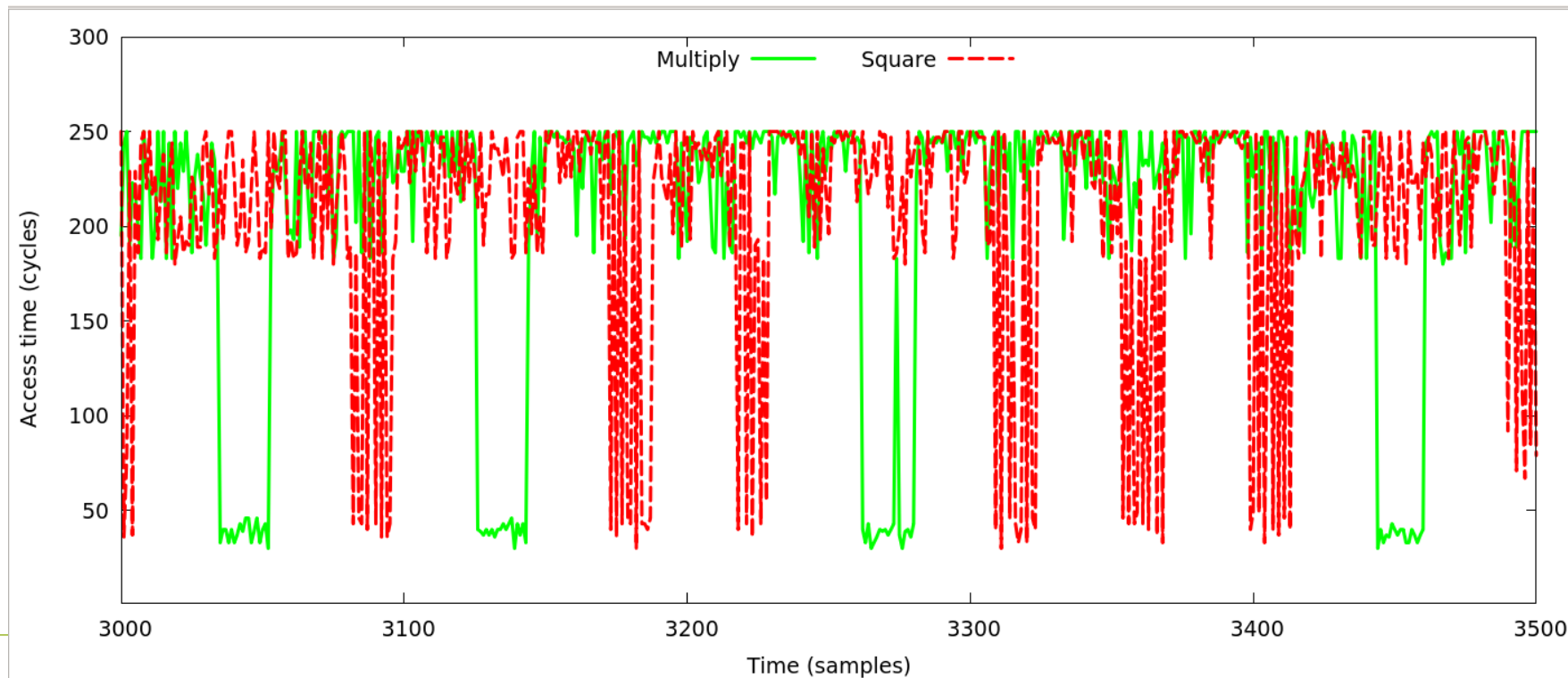
$11^5 \bmod 100 =$

$161,051 \bmod 100 = 51$

Operation	x	i	d_i
	1	2	101

**The private
key is
encoded in
the
sequence of
operations
!!!**

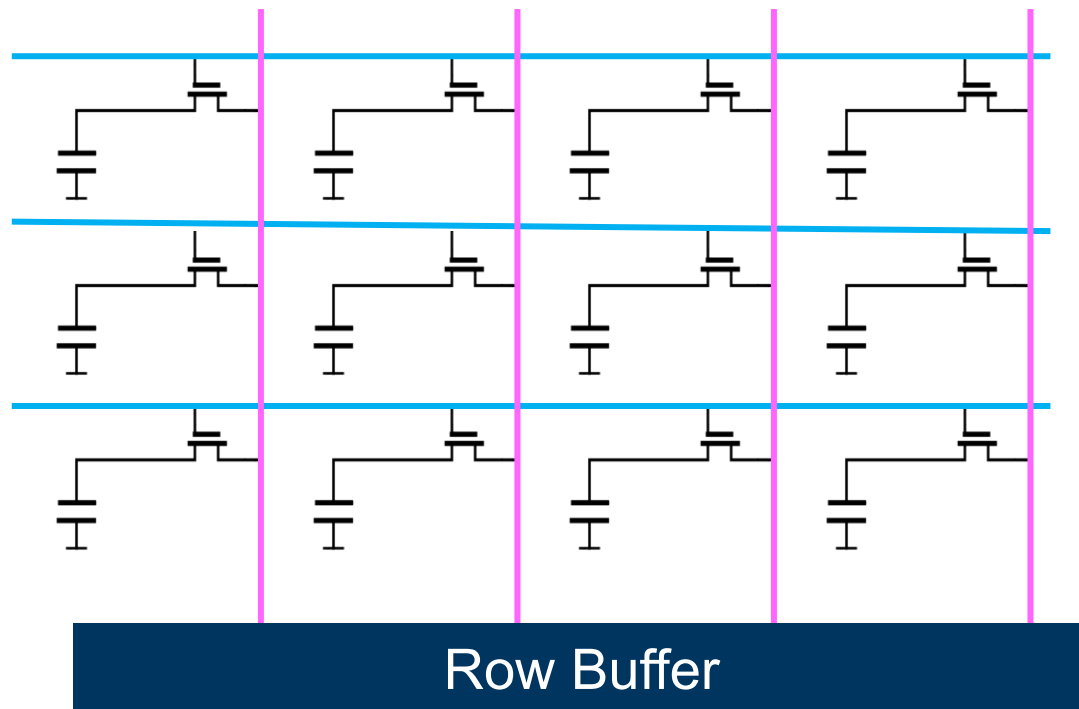
Flush+Reload on Square and Multiply



Problem: Integrity

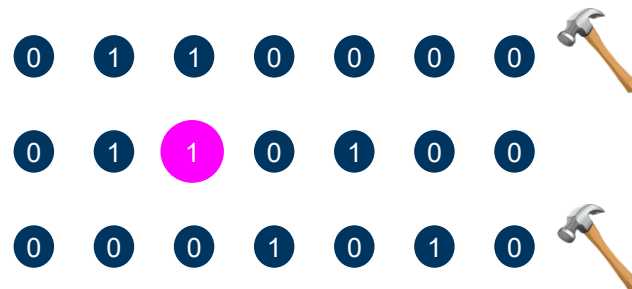
DRAM Array Organisation

- Activation: copies a row to the row buffer
- Read/write operate on row buffer
- Capacitors lose charge
 - Each row is refreshed at least once in 64ms



Rowhammer (Kim et al. ISCA 2014)

- Repeatedly toggling a wordline (aggressor row) causes disturbance errors in neighbouring rows
- Repeatabe
- One way flip
- Data dependent

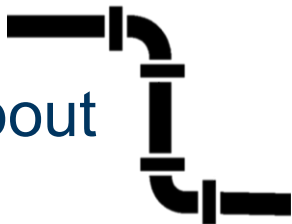


Problem: Performance

Challenges to (automatic) code generation

General

Reasoning about
the pipeline



- Registers are faster than memory

Crypto specific

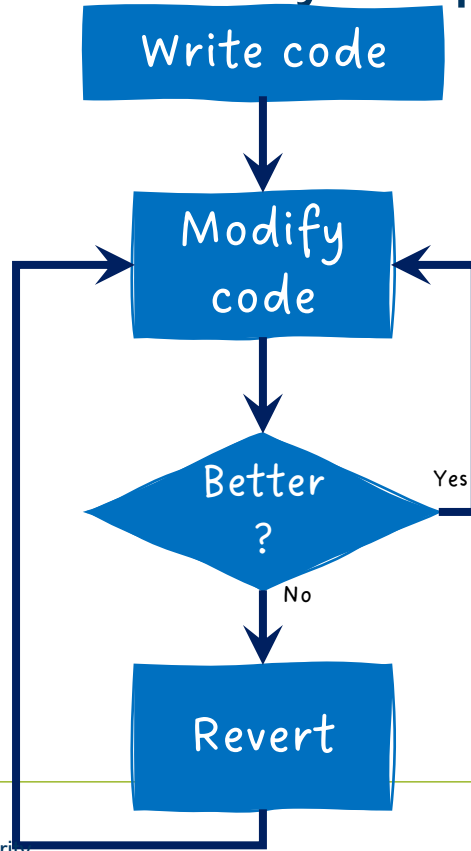
Simple code

- Unrolling
- No secret branches
- Simple memory access patterns



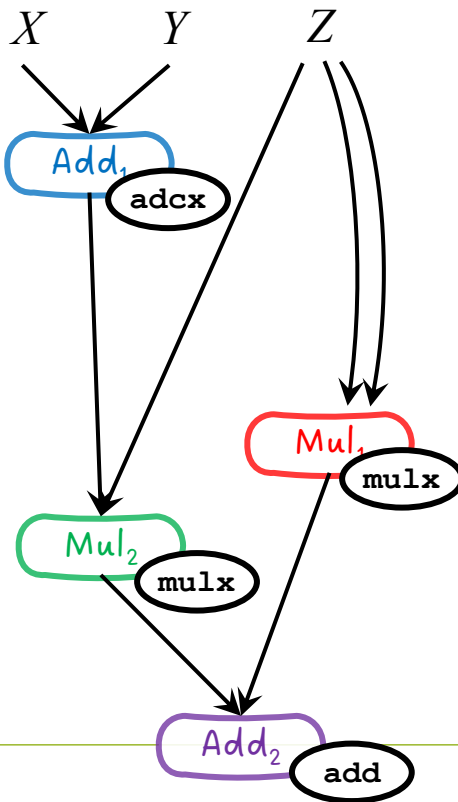
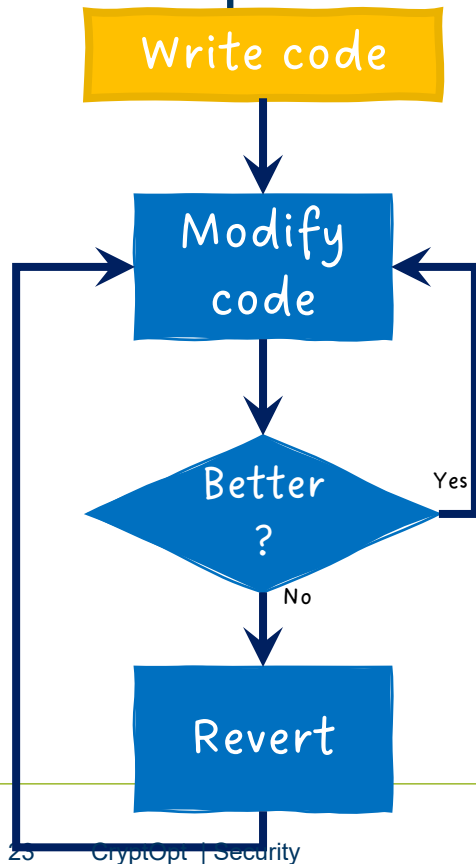
Large Basic Blocks

Approach – first year programming



“Professional” name:
Random Local Search

Example Function: $(X+Y)Z+Z^2$



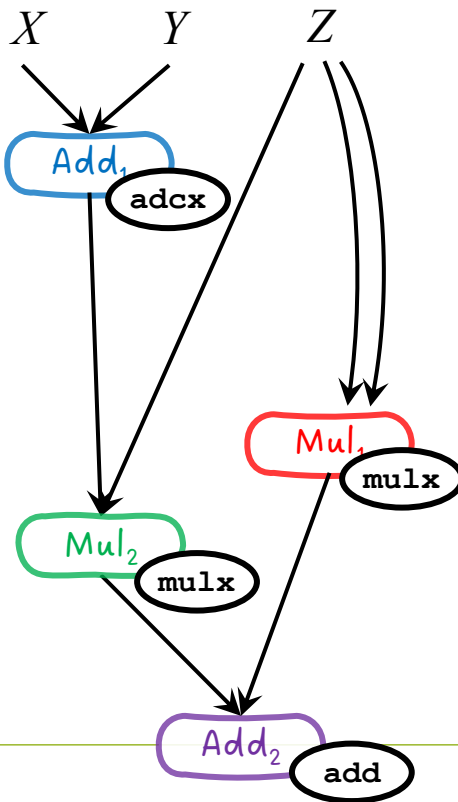
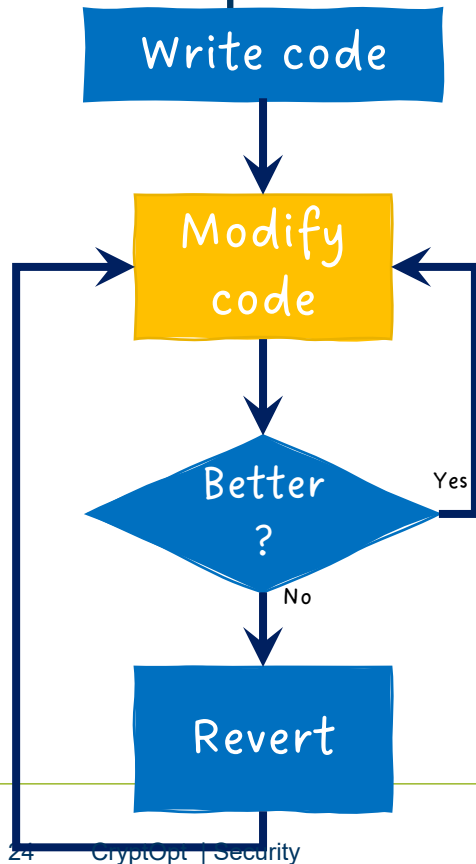
```
mov rax, [X]
clc
adcx rax, [Y]
```

```
mov rdx, [Z]
mulx r8, r9, rdx
```

```
mulx r10, r11, [Z]
```

```
add r11, r9
mov [out], r11
```

Example Function: $(X+Y)Z+Z^2$



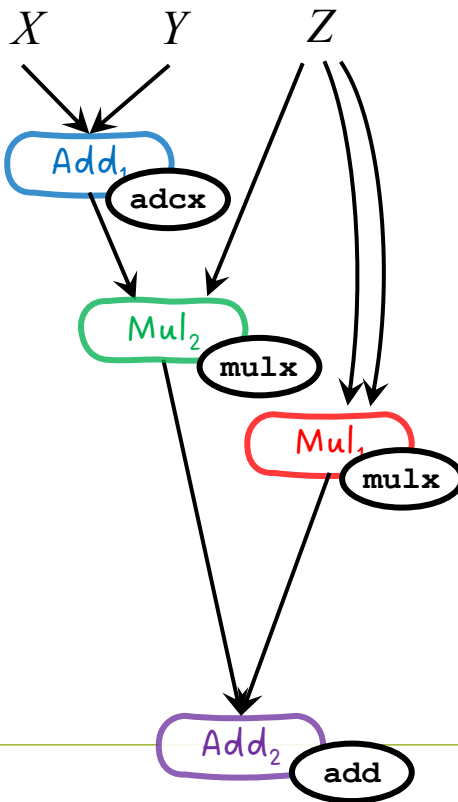
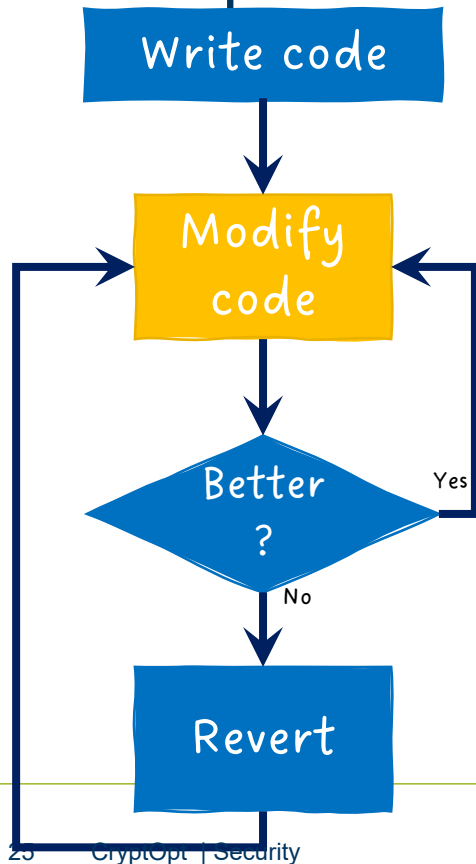
```
mov rax, [X]
clc
adc rax, [Y]
```

```
mov rdx, [Z]
mul r8, r9, rdx
```

```
mul r10, r11, [Z]
```

```
add r11, r9
mov [out], r11
```


Example Function: $(X+Y)Z+Z^2$



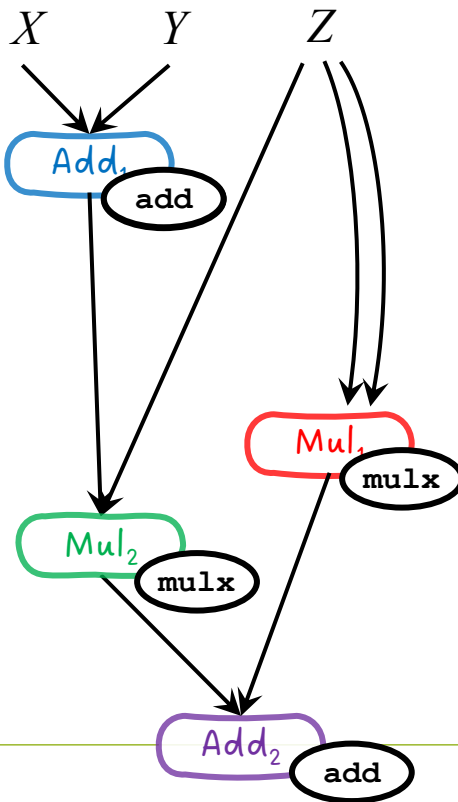
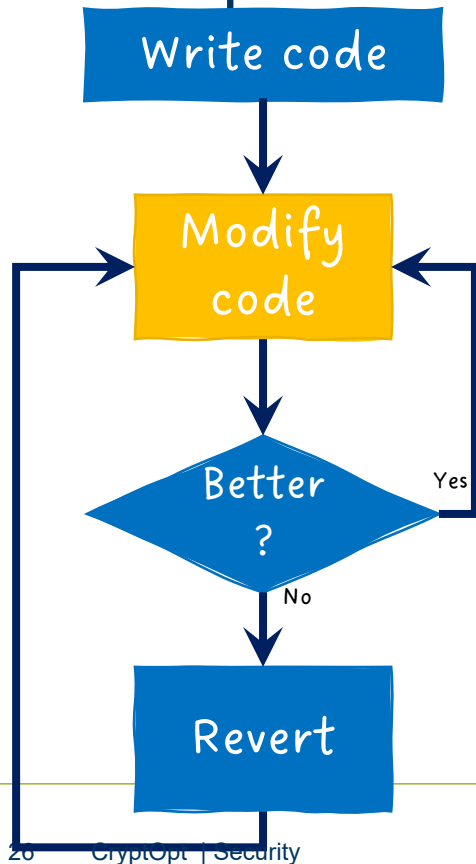
```
mov rax, [X]
clc
adcx rax, [Y]
```

```
mov rdx, [Z]
mulx r8, r9, rdx
```

```
mulx r10, r11, [Z]
```

```
add r11, r9
mov [out], r11
```

Example Function: $(X+Y)Z+Z^2$



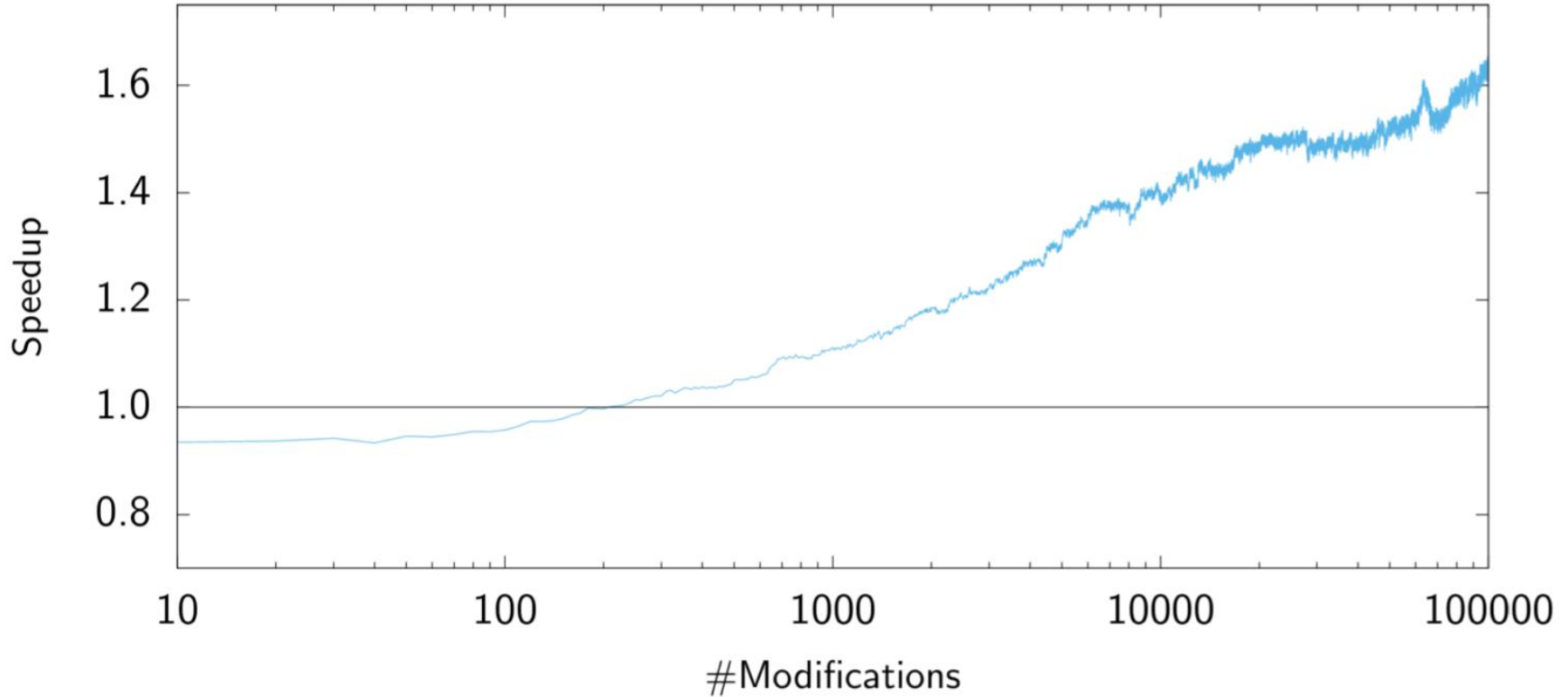
```
mov rax, [X]
ele
addrax, [Y]
```

```
mov rdx, [Z]
mulx r8, r9, rdx
```

```
mulx r10, r11, [Z]
```

```
add r11, r9
mov [out], r11
```

Results



Performance – field operations

	Multiply	Square
Curve		
Curve25519		
P-224		
P-256		
P-384		
SIKEp434		
Curve448		
P-521		
Poly1305		
secp256k1		

Want to know more?

Chair for Computer Security

- Cache Attacks
 - Bachelor practical course – next offering SuSe 2026
- Introduction to CPU Design
 - Bachelor course – next offering SuSe 2026
- Microarchitectural Attacks and Defenses
 - Masters course – next offering WiSe 2026/27
- Seminars
- Projects
- Theses

Summary

Hardware design plays an important role in system security

Multiple problems have been identified

Active research area